

Assignment 2: Turing Machines — Design, Analysis & Simulation

Name: Arqam Chinoy

ERP No.: 32498

Personalization block: $R = 98$ $V = 2$ $B = 1100010$ $n = 4$

GitHub repository: https://github.com/arqamchinoy/32498-ITC_Assignment2

ERP number give $R = 98$. $V = 98 \bmod 4 = 2$. In binary $98 = 64 + 32 + 2$, so $B = 1100010$. The digits of my ERP number 32498 add up to $3+2+4+9+8 = 26$, and $26 \bmod 5 = 1$, so $n = 1 + 3 = 4$.

With $V = 2$ my variants are: Q1 $M = 5$,

Q2 $a^{2^n}b^n$,

Q3 $\times 2$ (append a 0),

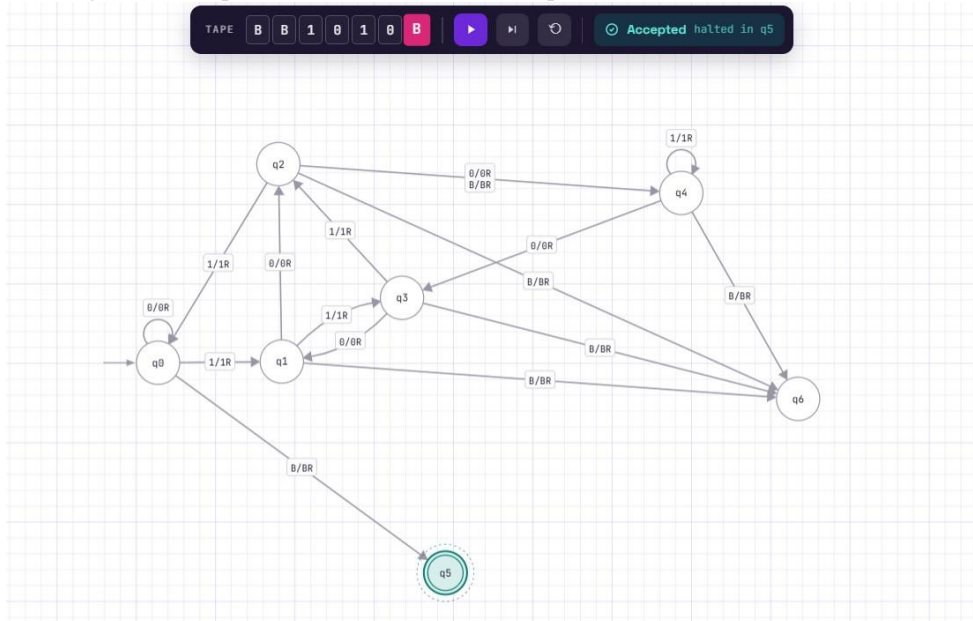
Q4 even-length palindromes over $\{a,b\}$,

Q5 $\{a^i \# b^j : i > j\}$.

Q1 — Divisible-by-M Recognizer ($V = 2$, $M = 5$, $B = 1100010$)

Task A

My machine is a remainder tracker for $M = 5$. It has five states q_0 to q_4 , and being in state q_r means the bits read so far form a number whose remainder mod 5 is r . When the next bit b is read, the number x becomes $2x + b$, so the new remainder is $(2r + b) \bmod 5$. The head only moves right and writes back the same symbol it read, so the tape is never changed. In q_1 to q_4 the blank sends it to the reject state q_6 . The empty string and strings of only zeros never leave q_0 , so they are accepted, which is what the question asks for 0.

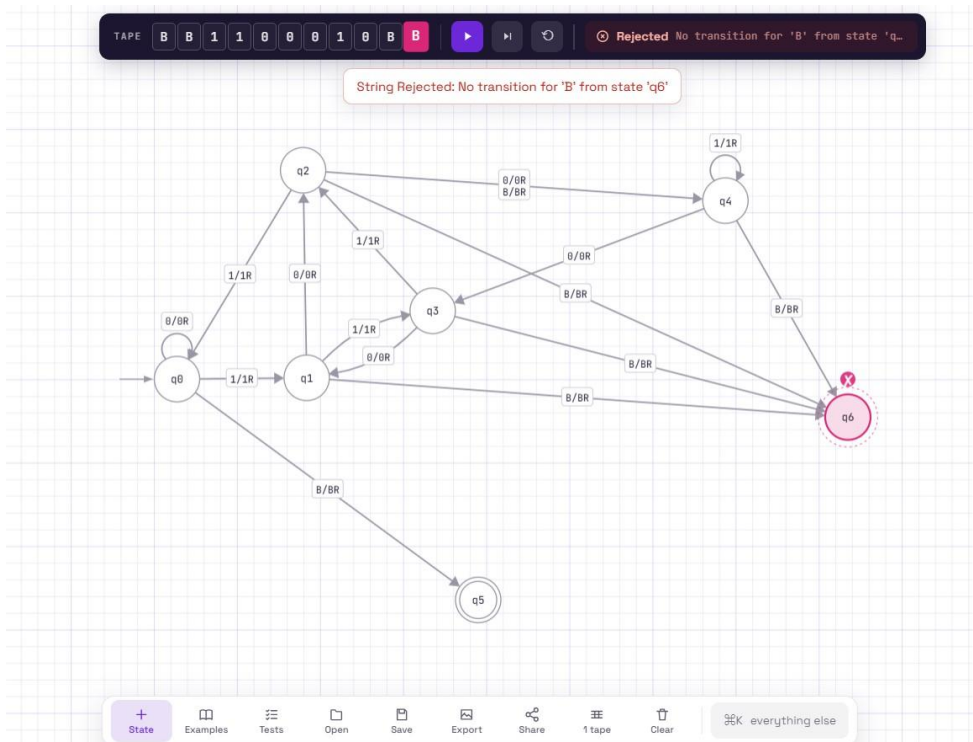


State diagram of the machine (screenshot from run 2).

Task C

(i) Run 1, input $B = 1100010$. <https://www.automataverse.com/m/sqgL2PLo>

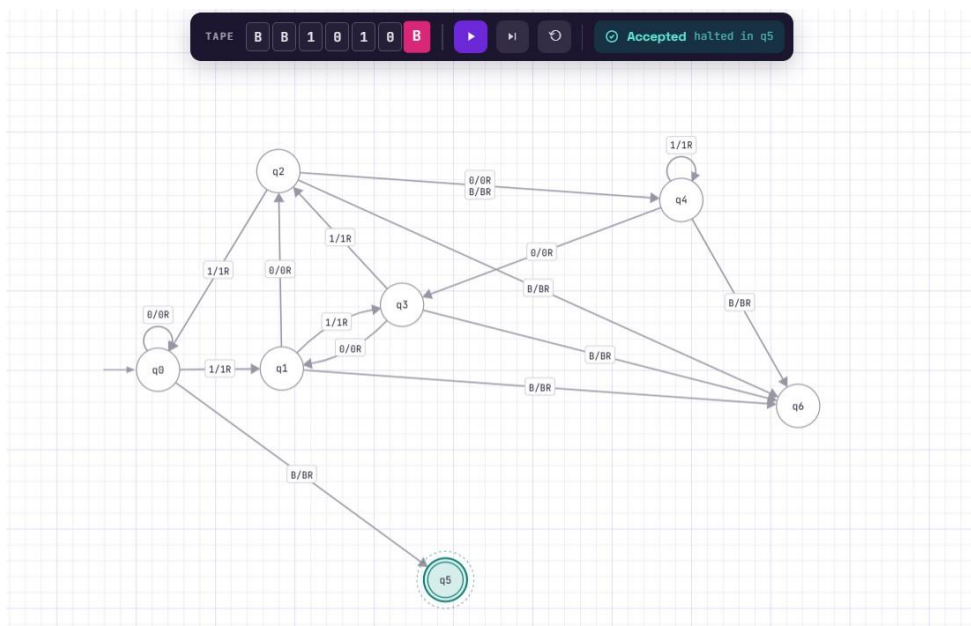
The machine rejected. 1100010 is 98 in decimal and $98 \bmod 5 = 3$, so rejecting is correct.



(ii) Run 2, input 1010 (this is 10, divisible by 5). Link:

<https://www.automataverse.com/m/vrpG3jLt>

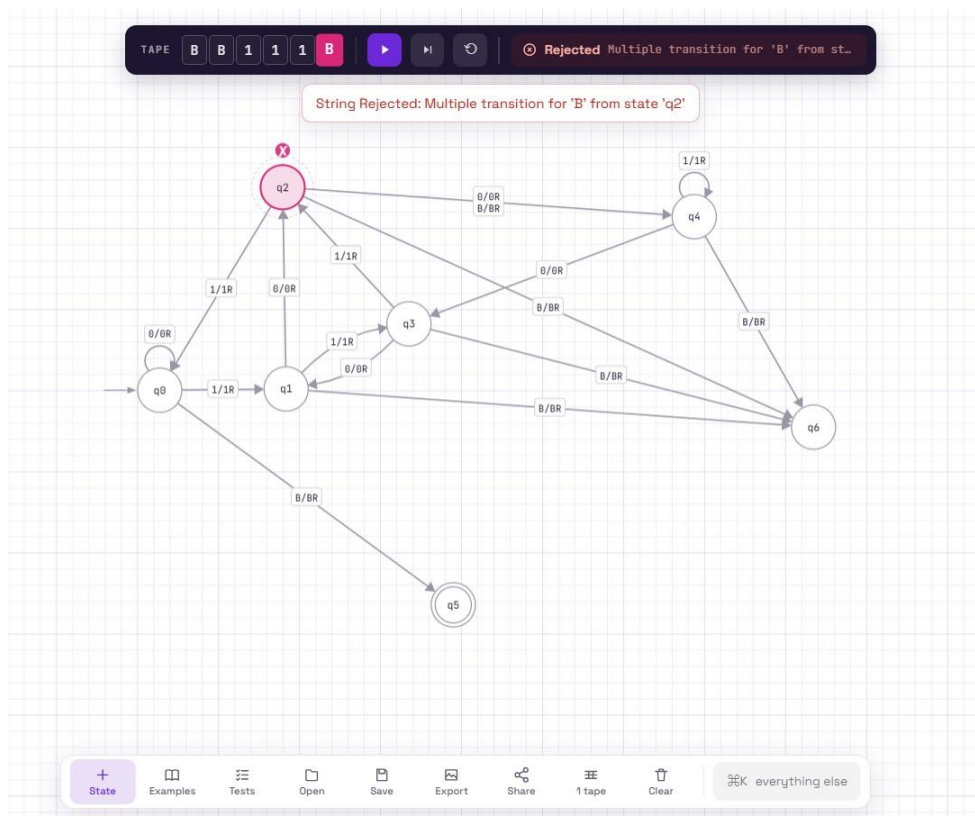
The machine accepted and halted in q_5 , which is correct since $10 \bmod 5 = 0$.



(iii) Run 3, input 111 (this is 7, not divisible by 5).

Link: <https://www.automataverse.com/m/YqXv6koK>

The machine rejected, which is correct since $7 \bmod 5 = 2$. (The message in the simulator for this run mentions the duplicate blank edge on q_2 mentioned under Task B; the verdict is the same either way.)



Task D

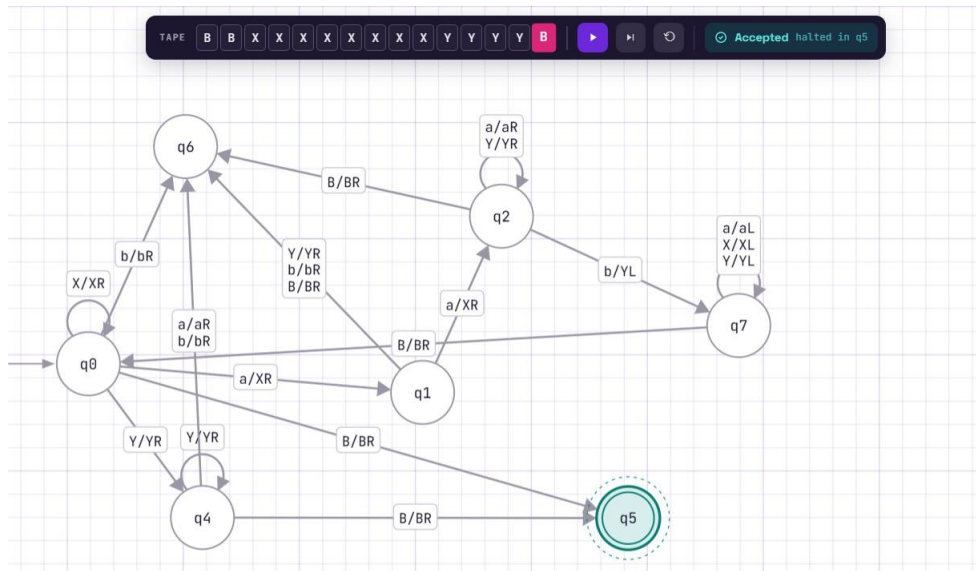
The five states of my machine are the five possible remainders mod 5, and every arrow was worked out from $(2r + b) \bmod 5$. For a different M I would need M states instead of five, and all the arrows would have to be worked out again with mod M (for $M = 3$ for example q_1 on 1 would go to q_0 , not to q_3). Only the acceptance rule, accept in q_0 at the blank, would stay same.

Q2 — Scaled-Count Recognizer ($V = 2$, language $a^{2n} b^n$, $n = 4$)

Task A

The language is every string of the form a 's followed by b 's where the number of a 's is exactly twice the number of b 's. In each round the machine crosses off two a 's from the left (q_0 turns the first a into X and q_1 turns the second one into X) and then walks right in q_2 , skipping over the remaining a 's and any Y 's, until it finds a b , which it turns into Y . Then q_7 walks back left to the blank at the start of the tape and the next round begins in q_0 . In q_0 the X 's are skipped. If q_0 meets a Y , no a 's are left and q_4 checks that only Y 's follow until the blank, and then it accepts in q_5 . That is what catches the strings with too many a 's.

Task B.



State diagram with the halted configuration of Run 1.

Task C

(i) Run 1, input $aaaaaaabbbb$ ($a^8 b^4$, which is $a^{2n} b^n$ with $n = 4$).

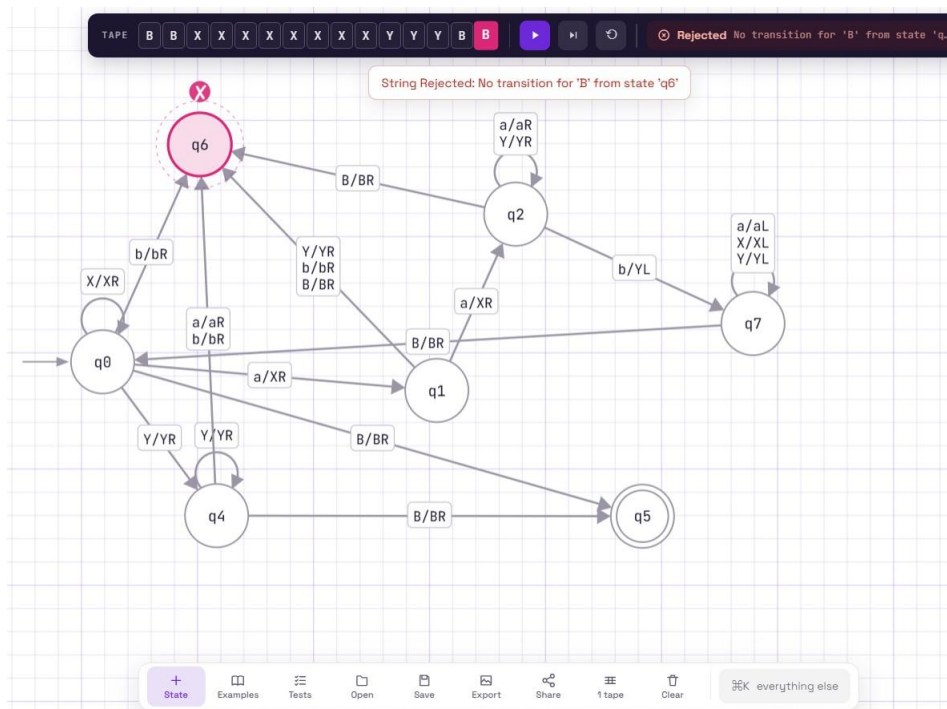
Link: <https://www.automataverse.com/m/3t56Zgpf>

Accepted, halted in q_5 . Correct.

(ii) Run 2, input $aaaaaaabbb$ ($a^8 b^3$, one b removed).

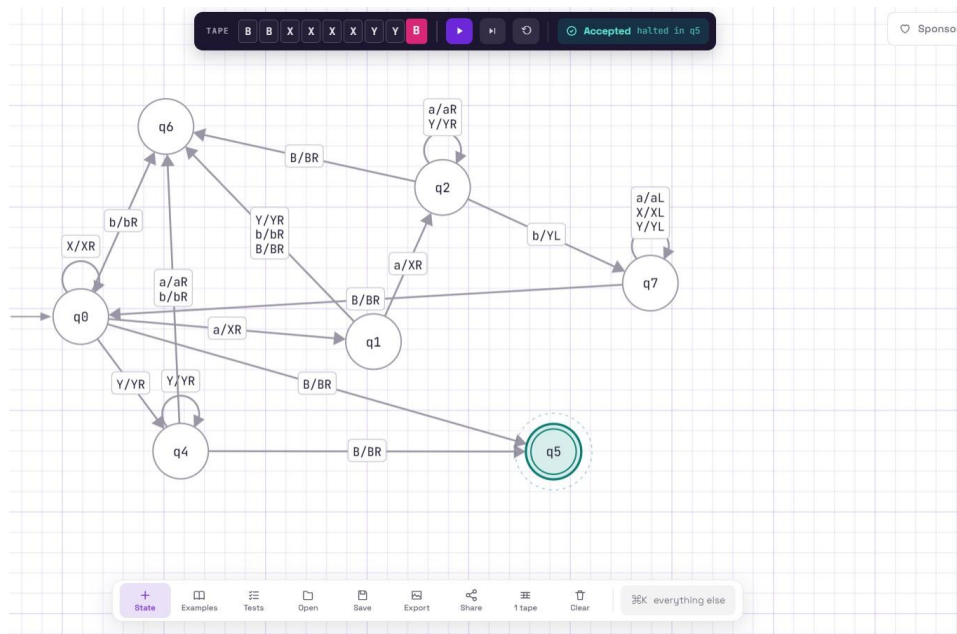
Link: <https://www.automataverse.com/m/ewp7FzFN> Rejected (no transition from q_6).

Correct, because 8 a 's need 4 b 's.

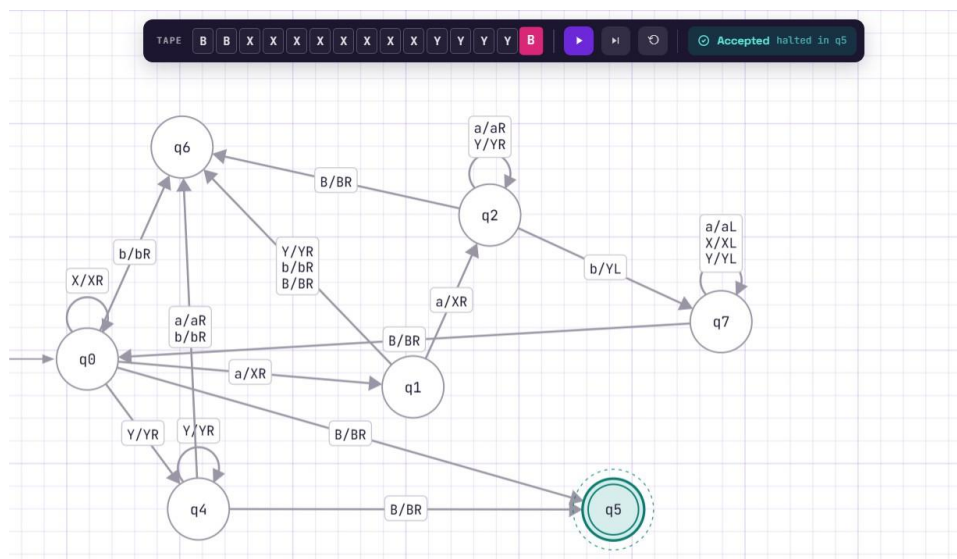


(iii) Run 3, input $aaaabb$ ($a^4 b^2$, the $n = 2$ case).

Link: <https://www.automataverse.com/m/gA3Qa3gU> Accepted, halted in q_5 . Correct.



The halted tape of Run 1 is below.



Task D

The place where I expected an off-by-one was the pair of states q_0 and q_1 , because a count of a 's that is odd (like $2n + 1$) slips through if the machine only crosses off one a per round. I made q_1 reject unless the very next symbol is another a , so each round removes exactly two a 's and one b , and a left-over a or an odd count always ends in q_6 . The other risk was q_4 , which has to accept only after the last Y and not when it sees a b , so a b after the Y 's is rejected there.

Q3 — Binary Transformer ($V = 2$, operation $\times 2$, $B = 1100010$)

Task A

Multiplying a binary number by 2 is the same as a left shift, which means putting a 0 at the right end of the string. So my algorithm is very short. In q_0 the head moves right over every 0 and 1 without changing them. When it reaches the first blank, which is just after the last bit,

it writes a 0 there, moves right and enters the halting state q1. I did not have to shift any bits, since the new 0 goes onto a cell that is already blank, and the result is longer than the input so there are no leftover blanks to clean up.

Task B

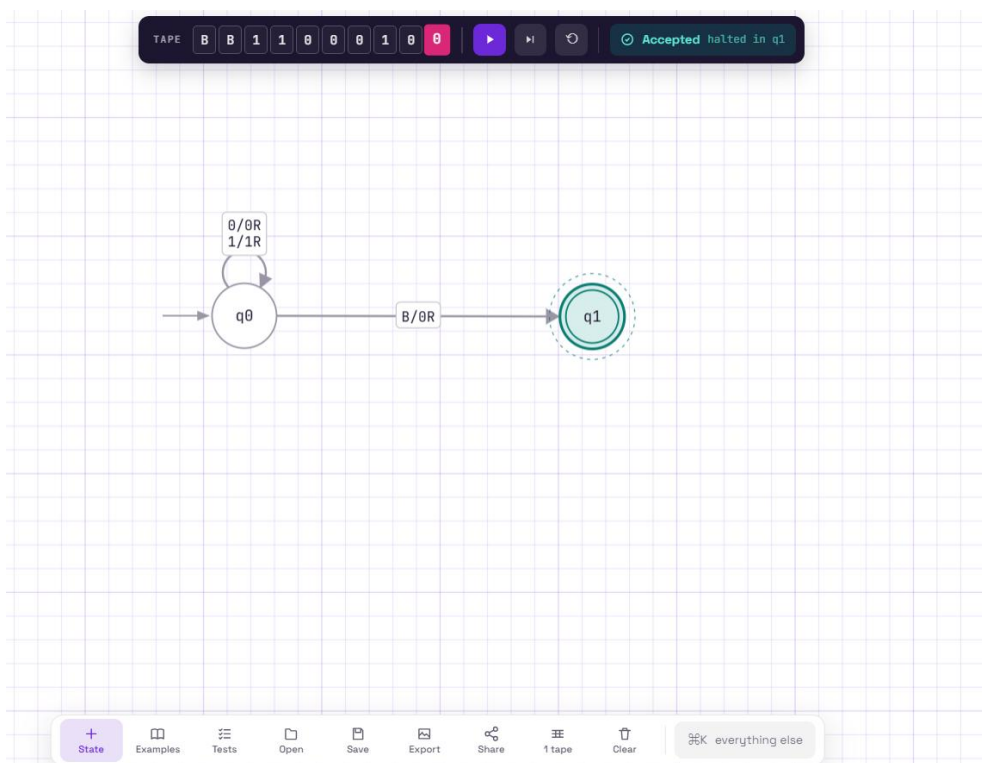
Current state	Read	Write	Move	Next state
q0	0	0	R	q0
q0	1	1	R	q0
q0	B	0	R	q1

States: q0 = scan to the right end (start), q1 = halt (final state).

Task C

Input (starting tape): B = 1100010.

Link: <https://www.automataverse.com/m/WCdLWe48>



Halted machine: the tape reads 11000100 and the machine stopped in q1.

By hand: $B = 1100010 = 64 + 32 + 2 = 98$ in decimal. The operation is $\times 2$, so the result should be $98 \times 2 = 196$. The final tape is $11000100 = 128 + 64 + 4 = 196$. They match.

Task D

The edge case is that the new bit has to go at the far right end, so the head has to travel across the whole string first, and I cannot just change something near the leftmost bit. If I only changed the leftmost bit of 1100010, I would get something like 0100010 = 34 which is

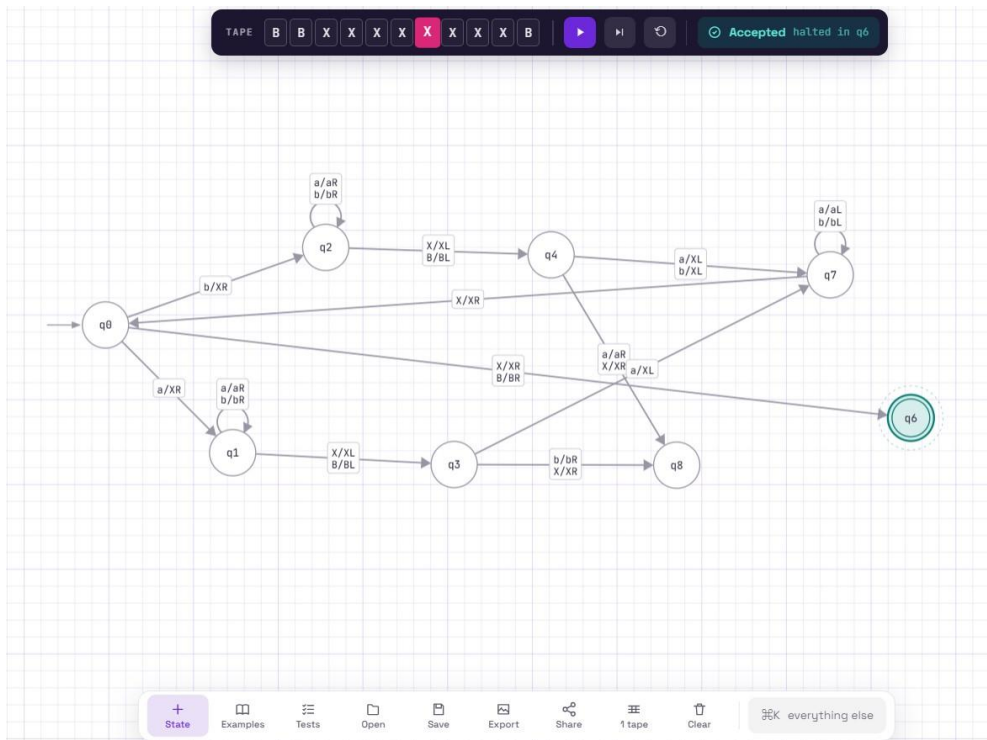
not even close to 196. Another case I checked is the input 0, which becomes 00 on my machine; the value is still 0 but it has a leading zero, and I did not add a separate cleanup step for this.

Q4 — Palindrome-Family Recognizer ($V = 2$, even-length palindromes over $\{a,b\}$, $n = 4$)

Task A

The machine matches the outermost symbols and works inwards. In q_0 it reads the leftmost unmarked symbol, replaces it with X, and remembers what it was by going to q_1 (it was an a) or q_2 (it was a b). Then it runs right to the end of the string, to the first X or blank, and steps one cell back onto the last unmarked symbol in q_3 or q_4 . If that symbol is the same letter it is also turned into X, and q_7 walks left to the X block and steps right back into q_0 for the next round. If it is a different letter the machine goes to q_8 , a dead state that rejects. When q_0 finally sees an X or a blank, everything has been matched and it accepts in q_6 .

Task B



State diagram of the machine (Run 1, halted in q6).

Task C

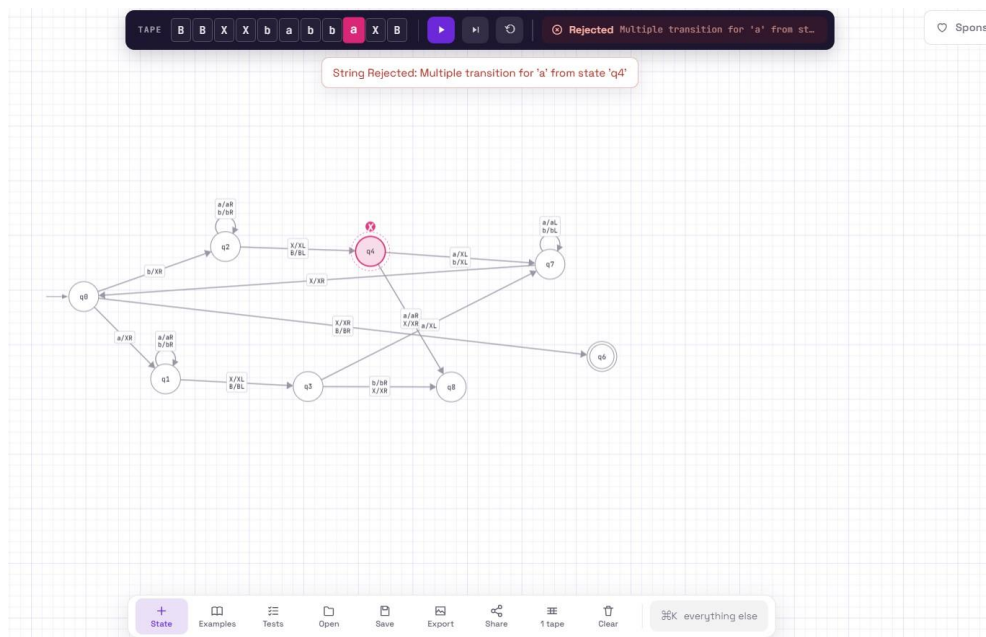
(i) Run 1, input abbaabba (length $8 = 2n$, an even palindrome).

Link: <https://www.automataverse.com/m/9BNNskth> Accepted in q6. Correct.

(ii) Run 2, input abbabbaa (length 8, not a palindrome, the reverse is aabbabba).

Link: <https://www.automataverse.com/m/c5tWXT9d>

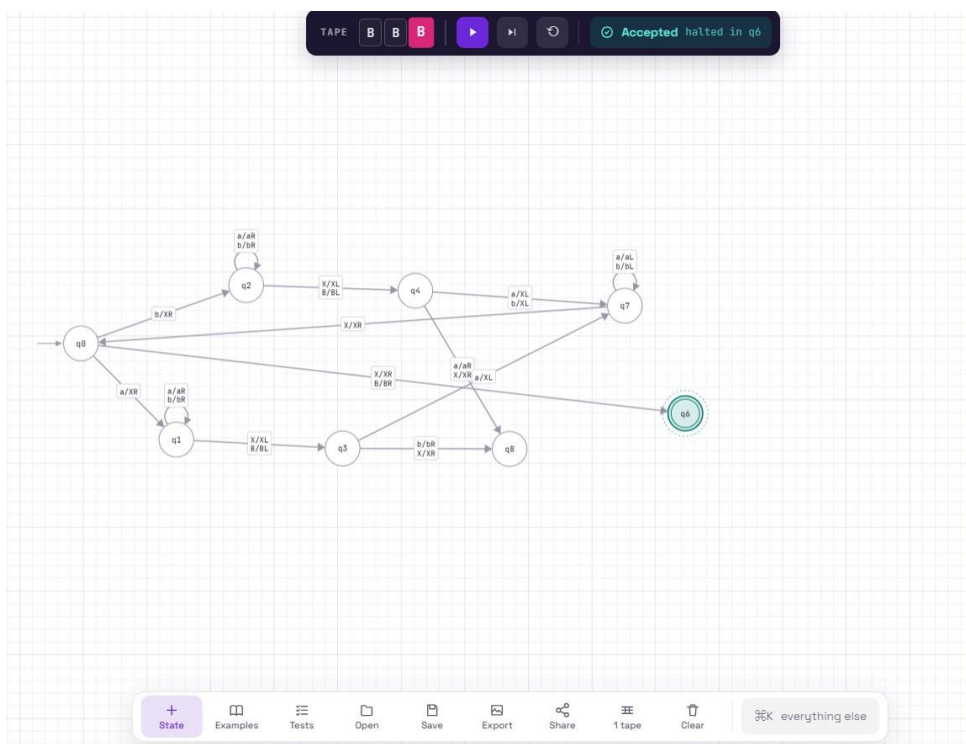
Rejected. Correct. The simulator stops with a message about multiple transitions for a from q4, which comes from the extra entry mentioned under Task B. The mismatch it was reporting is real (b on the left against a on the right).



(iii) Run 3, the empty string (the boundary case for even-length palindromes, length 0 is even).

Link: <https://www.automataverse.com/m/cWNrma5Z>

Accepted straight away because q0 sees a blank. Correct.



Task D

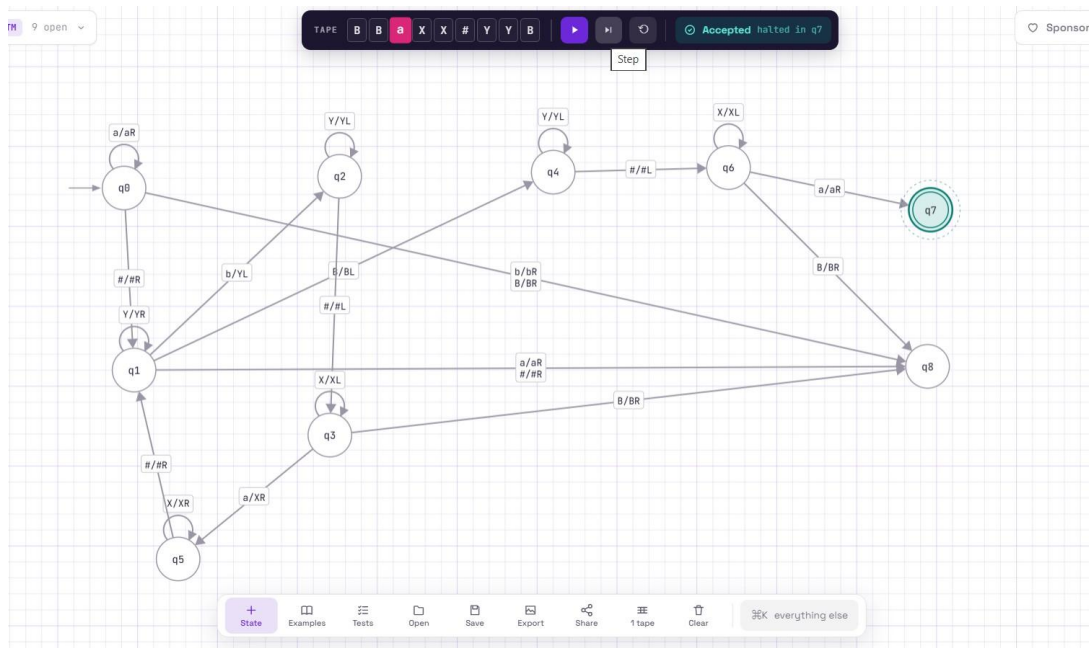
A single left-to-right pass would only let me read the string in the order it is written, but a palindrome check compares the first symbol with the last, the second with the second to last and so on. The head can only look at one cell at a time and a machine has finitely many states, so it can remember one symbol (my q_1 and q_2 do that) but not the whole first half. That is why I cross off one symbol, run to the other end, compare, and run back, and this has to be repeated for each pair, so the number of sweeps grows with the length (4 rounds for my length 8 string). With one pass only, the machine would either forget the left symbols by the time it reached the right side, or it could not tell which symbol on the right belongs to which on the left.

Q5 — Marked-Comparison Recognizer ($V = 2, \{a^i \# b^j : i > j\}, B = 1100010$)

Task A

The idea is to pair off one b on the right with one a on the left, round after round. In q0 the head skips the a's to find the #. In q1 it moves right over Y's (b's already used) and turns the first b it finds into Y. Then q2 goes back left over the Y's to the #, and q3 goes left over the X's until it finds the rightmost a that is not crossed off yet, and turns it into X. After that q5 goes right over the X's to the # and q1 takes over again. When q1 reaches the blank, all b's have been matched, and the head goes back (q4, then q6) over the Y's and X's to look at the left side. If it finds one more a that is not crossed off, there are more a's than b's, and it accepts in q7. If not, it rejects. If q3 runs out of a's while there is still a b to match, that means $i \leq j$, and it rejects in q8.

Task B



State diagram of the machine (Run 1, halted in q7).

Task C

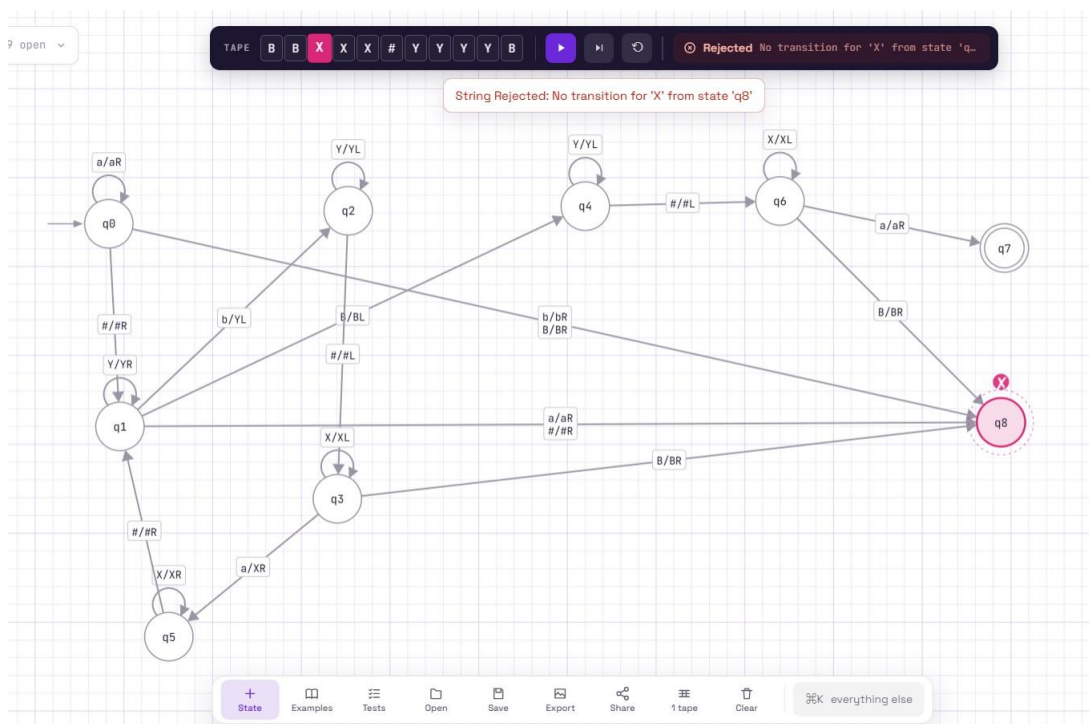
Conversion of B: B = 1100010 has three 1-bits and four 0-bits. I wrote one a for each 1-bit, so the left side of the # is aaa. The language only has a's on the left of the #, so the 0-bits (which would be b's) are not written on the left side.

(i) Run 1, input aaa#bb (i = 3, j = 2, so i > j, a member).

Link: <https://www.automataverse.com/m/NBSW5aUQ> Accepted, halted in q7. Correct.

(ii) Run 2, input aaa#bbbb (i = 3, j = 4, not a member).

Link: <https://www.automataverse.com/m/PNUSXMYw> Rejected (no transition for X from q8). Correct.



Task D

Whether i is bigger than j depends on the total count of a 's and of b 's, and the machine has no counter, only a fixed number of states, while i can be any size. When the head is at the b 's on the right it has already moved past the a 's, so the information about how many there were is only on the tape, and I have to go back to the left to use it. This is why my machine must go back and forth: for each b I go back left over the X 's to cross off one more a , and then go right again to find the next b . In one left-to-right pass the machine would have to store the number of a 's in its state, which is not possible for an unbounded i .

Q6 — Debug the Broken Machine

In the traces below the blank is written `_`, and the head position is shown by square brackets around the cell it is on. The tape is shown from cell 0.

Task A

Input `""` (empty string). The tape is all blanks, the head starts on cell 0.

Step	State	Tape	Head	Rule used
0	q_even	[_]	cell 0	start
1	q_odd	_ [_]	cell 1	q_even, _ → write _, R, q_odd
2	q_reject	_ _ [_]	cell 2	q_odd, _ → write _, R, q_reject. HALT, rejected

Input `"11"`.

Step	State	Tape	Head	Rule used
0	q_even	[1] 1 _	cell 0	start
1	q_odd	1 [1] _	cell 1	q_even, 1 → 1, R, q_odd
2	q_even	1 1 [_]	cell 2	q_odd, 1 → 1, R, q_even
3	q_odd	1 1 _ [_]	cell 3	q_even, _ → _, R, q_odd (row 3 of the table)
4	q_reject	1 1 _ _ [_]	cell 4	q_odd, _ → _, R, q_reject. HALT, rejected

Task B

Both traces ended in `q_reject`, but both inputs have an even number of 1s (zero and two) so both should be accepted. In fact, `q_accept` was not reached in either trace, and looking at the table, no row has `q_accept` as its next state at all, so the machine can never accept any string. The difference between the two traces is useful: in the trace of `"11"` the machine was in `q_even` at the end of the input (after step 2, with two 1s read), which is the correct state for an even count, and in the same way `""` is in `q_even` at the end. At that moment it is reading a blank in state `q_even`, and the rule that fires is row 3, which sends it to `q_odd` and then to `q_reject`. So the wrong row is row 3: `q_even`, blank → blank, R, `q_odd`.

The correct entry is `q_even`, `□` → `□`, R, **`q_accept`**. Reading a blank in `q_even` means the input ended with an even number of 1s, and this should be an accept. Row 6 (`q_odd`, blank → `q_reject`) is correct as it is, since the blank was reached after an odd count. The rest of the rows only track parity (0 keeps the state, 1 flips it), and the traces used rows 1, 2 and 5 correctly.

Task C

I made two machine files for this question, the broken one exactly as given and the corrected one. State names in the simulator: `q0` = `q_even`, `q1` = `q_odd`, `q2` = `q_accept`, `q3` = `q_reject`. The files are in the `q6` folder of the repository (`q6_broken_empty.json`, `q6_broken_11.json`, `q6_broken_1.json`, `q6_broken_111.json` and the same four with `fixed` instead of `broken`). The screenshot of each halted run is saved next to its JSON file under the same name, and the shareable links are in `README.md` (lines `q6 run 1` to `q6 run 8`).

Input	Broken machine	Fixed machine	Correct answer
<code>""</code> (empty)	Reject (wrong)	Accept	Accept
<code>11</code>	Reject (wrong)	Accept	Accept
<code>1</code>	Reject	Reject	Reject
<code>111</code>	Reject	Reject	Reject

Task D

The strings "1" and "111" alone would not have found the bug. They have an odd number of 1s, so they should be rejected, and the broken machine rejects everything, so it looked correct on them. Only the strings with an even count (" " and "11") showed the problem, since they are the ones that must reach the accept state. So I learned that a test set must include inputs from both sides of every decision the machine makes, and especially cases that should be accepted, because a machine that never accepts passes all the reject tests. The empty string is also a good boundary case to always include.